



中山大學
SUN YAT-SEN UNIVERSITY

多模态大模型原理与应用课程报告

基于视觉理解与自动操作的智能浏览器助手

姓名 _____

学号 _____

学院 _____ 计算机学院

专业 _____ 计算机科学与技术

目录

一 应用背景与任务定义	2
1 应用背景	2
2 任务定义	3
二 相关工作调研	3
1 多模态网页 Agent	3
2 视觉定位与混合感知	4
3 相关开源项目	4
三 系统方案设计	4
1 SoM 视觉输入模式	4
1.1 感知阶段 (Perceive)	5
1.2 推理阶段 (Reason)	7
1.3 执行阶段 (Act)	8
1.4 验证阶段 (Verify)	8
1.5 前端 Web 交互界面	9
2 HTML 文本输入模式	10
四 实验评测与分析	11
1 HTML 输入模式下的性能测评	12
2 SoM 输入模式下的性能测评	13
五 参考文献	14

一 应用背景与任务定义

1. 应用背景

随着互联网的快速发展，网页的样式和信息量都在经历日新月异地增长，为人们完成特定的网页任务带来了挑战。在多模态大模型技术成熟的背景下，基于 VLM/LLM 技术实现的多模态网页 Agent 能够实现辅助和某种程度地替代人类完成对应的网页任务，具有广阔的应用前景。Web Agent 可以作为自动化测试工具，辅助软件开发团队进行前

端的冒烟测试与回归测试。**Web Agent** 也可以作为智能个人助理，帮助个人用户执行复杂任务，降低网页任务的操作成本。除此以外，**Web Agent** 可以实现复杂数据的收集与分析、改进针对障碍人士的辅助功能等任务。

2. 任务定义

我们计划实现一个能够理解自然语言指令，并通过浏览器自主完成复杂任务的智能代理。用户将网页任务目标作为输入，在执行任务的每一步中获取网页的 **Accessibility Tree** 并根据网页截图进行 **SoM** 标记，将网页截图与 **SoM** 输入多模态大模型，生成对应的指令，并使用 **playwright** 执行并等待页面反馈、进行任务的下一步。基于任务导向，我们计划实现：

- 一个 **SoM** 内容标注器，根据网页截图和 **Accessibility Tree** 生成元素边界框和对应的 **id**，提高模型的准确率
- 一个基于 **VLM** 的 **Web Agent**，根据用户输入进行任务的分步计划、在每一步根据网页截图信息和 **SoM** 标注生成对应的网页操作指令
- 一个 **playwright** 执行器，解析 **Agent** 输出的执行指令并执行，刷新网页进行下一步任务

二 相关工作调研

本课题相关工作涉及多模态 **GUI** 代理、网页自动化框架、任务规划与视觉定位等方向，以下列举代表性论文与开源项目。

1. 多模态网页 Agent

- **WebVoyager** [1]: 提出基于 **GPT-4V** 的端到端网页 **Agent**，在真实网站上通过截图和 **HTML** 辅助实现高成功率，验证了大视觉模型在网页导航中的可行性。
- **Fara-7B** (Microsoft Research) [2]: 一个 **7B** 参数的多模态模型，结合 **Magentic-UI** 框架，使用 **Playwright** 执行操作，在控制复杂度的同时实现良好性能，是本课题的核心参考。

- **WebAgent** 综述 [3]: 对基于 LLM 的网页自动化 Agent 进行了系统分类, 总结了规划、感知、执行的典型架构。

2. 视觉定位与混合感知

- **SoM (Set-of-Mark)** [4]: 通过为可交互元素叠加数字标记, 将 GUI 定位任务转化为标记选择任务, 显著提升 VLM 在界面上的 grounding 准确率。本系统将 SoM 作为预处理模块, 为 VLLM 提供可直接引用的标记索引。
- **R-VLM** [5]: 提出 IoU 感知的目标函数和缩放区域候选框, 将 GUI 元素定位精度大幅提升, 在 Mind2Web 基准上获得高准确率, 为改进视觉定位提供了理论支持。
- **WebGUM** [6]: 证明同时利用 HTML/可访问性树和视觉信息比纯视觉或纯 DOM 方法更有效, 为本课题混合感知策略提供了实验依据。

3. 相关开源项目

- **Playwright** [7]: 现代浏览器自动化框架, 提供跨浏览器、可靠的选择器和等待机制, 本课题的执行层核心。
- **OpenClaw / Nanobot** [8]: 开源的多模态 Agent 实现, 展示了从截图到操作的简易流水线, 但缺乏精心设计的错误恢复和混合感知。
- **SMART-LLM** [9]: 多 Agent 协作框架, 用于复杂任务规划, 可借鉴其任务分解与验证机制。

三 系统方案设计

1. SoM 视觉输入模式

SoM (Set-of-Mark) 模式是系统的主要运行模式。该模式将网页截图与可交互元素的视觉标记相结合, 使 VLM 能够直观地看到页面上的可操作元素, 并通过引用标记编号来精确指定操作目标。其整体架构如图1所示, 遵循 ReAct (Reasoning + Acting) 范式, 每步操作分为感知 (Perceive)、推理 (Reason)、执行 (Act) 和验证 (Verify) 四个阶段。

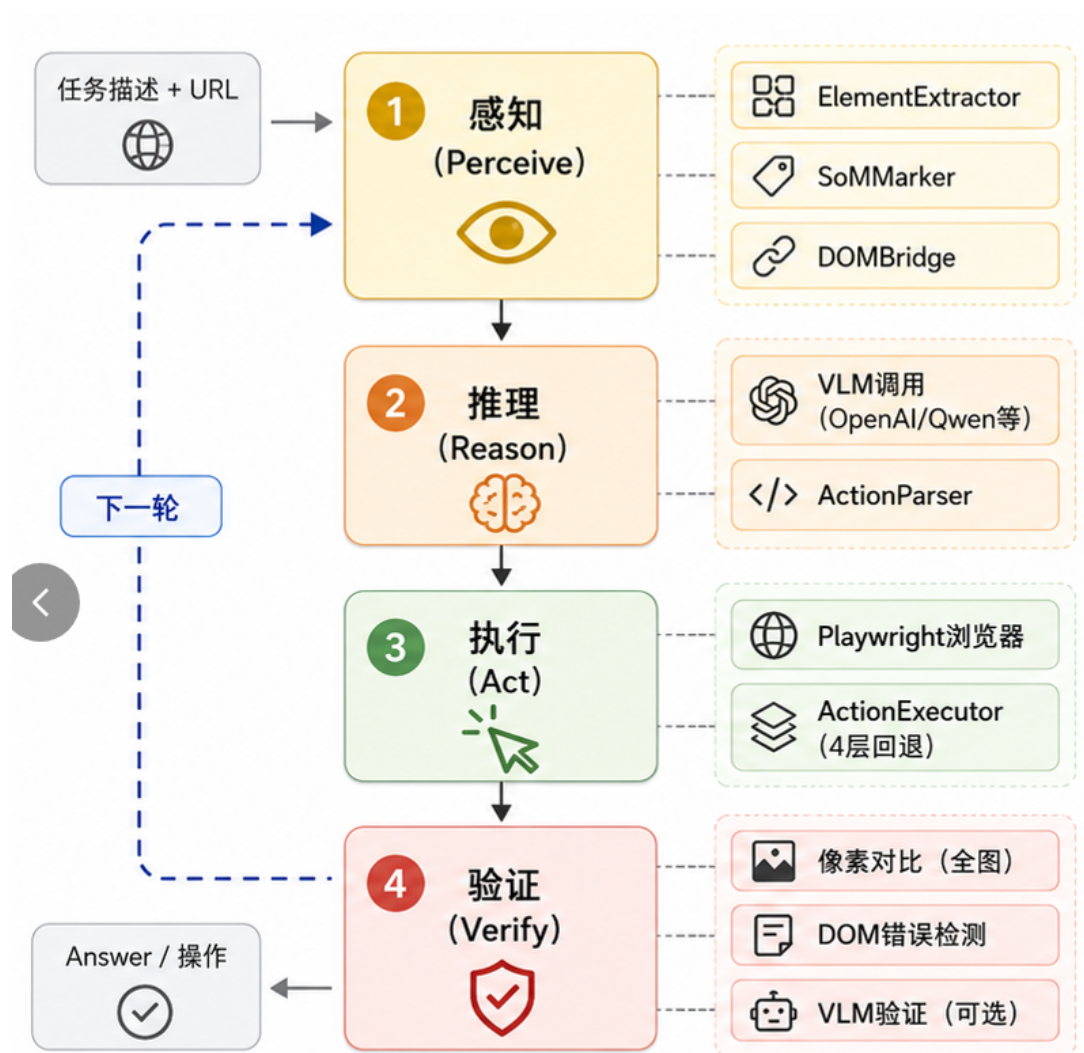


图 1: SoM 视觉输入模式的系统架构

1.1 感知阶段 (Perceive)

感知阶段负责将当前网页的视觉状态转化为 VLM 可以理解的结构化表示。该阶段的输入为 Playwright 控制的浏览器页面，输出为 SoM 标注后的截图以及可交互元素列表。

ElementExtractor 一可交互元素提取 ElementExtractor 模块通过 CSS 选择器扫描页面 DOM，提取所有可交互元素。主选择器覆盖了 HTML 原生交互标签（<button>、<a>、<input>、<select>、<textarea>）、WAI-ARIA 角色属性（[role="button"] 等）、事件处理器标记（[onclick]、[tabindex]）、无障碍标签（[aria-label]）以及 QQ 邮箱自定义的无障碍属性（[data-a11y]）。

```
INTERACTIVE_SELECTOR = (  
    "button, a, input, select, textarea, "  
    "[role='button'], [role='link'], [role='checkbox'], "  
    "[role='radio'], [role='combobox'], [role='listbox'], "  
    "[role='menu'], [role='tab'], [role='switch'], "  
    "[onclick], [tabindex], "  
    "[aria-label], [data-a11y]"  
)
```

Listing 1: INTERACTIVE_SELECTOR 定义

对于采用 React、Vue 等前端框架构建的 SPA 页面，按钮元素通常为纯 `<div>` 或 `<span>` 标签，不含任何上述可匹配属性。此时提取模块启动补充扫描：遍历页面所有元素，检测其计算样式中 `cursor` 属性是否为 `pointer`，并结合可见文本长度（1–8 字符）识别为框架按钮。这一启发式策略能够有效捕获纯 `div` 按钮。

所有候选元素经过可见性过滤（`display:none`、`visibility:hidden`、`opacity:0`）、尺寸过滤（最小 `4px2`）、交互性过滤（`disabled`、`pointer-events:none`）后，按照视觉位置排序并分配连续编号。同时，提取阶段会存储每个候选元素的 DOM 引用（`_el`），供后续 `data-som-id` 注入使用。

SoMMarker 一视觉标记绘制 SoMMarker 模块使用 PIL（Python Imaging Library）在页面截图上绘制交互元素的视觉标记。为每个候选元素绘制指定颜色的矩形边框（默认线宽 `2px`），并在边框上方或内侧放置带纯色背景的数字标签。标签采用 12 种高对比度颜色循环分配，相邻元素尽量使用不同颜色以便于区分。当标签因元素密集而可能重叠时，模块自动将标签移至下一个非重叠候选位置（边框上方、边框内侧左上角），确保所有标签清晰可读。

DOMBridge 一编号到 DOM 的双向映射 DOMBridge 维护 SoM 标记编号与 DOM 元素之间的双向映射关系。每个标记编号被映射至三个关键信息：元素的 CSS 选择器（用于 Playwright 定位）、元素的归一化边界框（用于坐标回退点击）以及后端节点 ID（用于 Mind2Web 评测）。在执行阶段，通过 `get_bbox(id)` 可获取指定编号元素的边界框，`som_id_to_selector(id)` 可将编号转换为 Playwright 定位器，支撑了从 VLM 文本输出到浏览器操作的完整链路。

1.2 推理阶段 (Reason)

推理阶段将感知阶段的结构化输出组装为 VLM 可理解的 prompt，调用模型获取下一步操作决策。

Prompt 组装 `build_som_user_prompt` 函数负责组装 user prompt，其结构包括：（1）用户任务描述；（2）当前页面 URL 与标题；（3）当前计划（从上一步推理输出中提取，帮助模型保持任务连贯性）；（4）最近 5 步的历史操作记录，以 [OK]/[NO EFFECT]/[FAIL] 前缀区分执行结果；（5）可交互元素文字列表，每行格式为 [编号] < 标签 > " 文本内容" aria=" 无障碍标签"，供模型与截图中的视觉标记交叉参照；（6）操作指令。完整的 prompt 由 system prompt（包含角色设定、响应格式、可用动作表、规则）与上述 user prompt 拼接而成。

登录与验证码的人机协同机制 现今大量网页需要进行登录或验证才能体验完整功能，而 agent 本身难以独立完成验证码识别和登录流程，因此一定场景下，人工的介入是有必要的。

系统采用“自动检测 + VLM 视觉判断”双层策略应对登录和验证码场景。程序化检测层在感知阶段运行，通过 JavaScript 扫描页面 DOM 中的密码输入框 (`input[type="password"]`)、扫码登录关键词（“微信登录”“QQ 登录”“二维码登录”等），快速识别登录页面。检测到后，Agent 直接暂停并通知前端等待人工介入。

VLM 视觉判断层作为补充，通过 System Prompt 中的 CAPTCHA / LOGIN —STOP IMMEDIATELY 规则引导：当 VLM 在截图或元素列表中识别到验证码（滑块、拼图、“安全验证”）、登录表单或扫码界面时，输出 captcha 或 login 操作。这两种操作被定义为终端操作 (`is_terminal=True`)，执行后跳过验证阶段，直接进入暂停等待。用户完成登录或验证，点击继续后，Agent 重新感知页面并恢复任务循环。

VLM 调用与响应解析 推理模块通过 ReasonerFactory 根据配置的 provider 名称 (qwen/ope-nai/anthropic/local) 创建对应的推理器实例。VLM 返回的 JSON 经 ActionParser 解析为 Action 数据类，包含操作类型 (ActionType 枚举)、目标元素编号和操作值（如待输入文本），交由执行器处理。

1.3 执行阶段 (Act)

执行阶段将 VLM 的决策转化为浏览器实际操作，并处理操作后的页面状态变化。

ActionExecutor 一多层回退点击 ActionExecutor 负责执行 VLM 输出的操作指令。对于 CLICK 操作，采用 4 层回退机制以确保最大的执行可靠性：

1. **Playwright 原生点击**：产生浏览器原生事件，最接近真实用户操作，Spa 与反爬检测系统无法区分。
2. **JS MouseEvent 派发**：通过 `dispatchEvent` 派发完整的 `mousedown/mouseup/click` 事件序列，同时调用 `el.click()` 触发默认行为。
3. **Href 导航**：对于 `<a>` 标签元素，提取其 `href` 属性并通过 `page.goto()` 直接导航。此机制专为绕过 Bing 等网站的点击拦截设计——Bing 在搜索结果链接上使用 `preventDefault()` 拦截点击事件，直接导航可完全规避该问题。
4. **坐标回退**：通过 `getBoundingClientRect` 获取实时元素坐标，滚动至该位置后执行 Playwright 屏幕坐标点击。

对于 TYPE 操作，执行器强制要求先通过 JS `focus()` 和 `select()` 将焦点赋予目标输入框，清除已有内容后再通过 Playwright 键盘输入新文本。

新标签页检测与同步 执行 CLICK 或 PRESS 操作后，系统检测浏览器上下文中的页面数量变化。若检测到新打开的标签页，自动将当前操作页面切换至最新标签页，并在 `screenshot()` 和 `get_page_url()` 等所有页面访问方法中通过 `context.pages[-1]` 确保始终操作最新标签页。

1.4 验证阶段 (Verify)

验证阶段在执行完成后立即对操作效果进行评估，为下一轮推理提供反馈信息。

全图像素对比 通过 PIL 的 `ImageChops.difference` 对操作前后的页面截图进行逐像素相减，统计差异像素比例。若差异像素少于 0.05%（约 460 像素），直接判定操作无效，跳过后续验证步骤。该方法采用 JPEG 格式降低解码延迟，使前端在操作完成后立即看到前后对比截图。

DOM 错误检测 在像素对比之后（无论其是否触发），通过 JavaScript 扫描页面 DOM 中的错误提示元素。检测目标包括：WAI-ARIA 标准角色（`[role="alert"]`、`[role="alertdialog"]`）、QQ 邮箱自定义 dialog 组件（`.xmail-ui-dialog`）、通用错误 toast 和表单验证消息。该方法利用错误提示必然出现在 DOM 中的特性，以极低成本覆盖了 VLM 可能遗漏的错误弹窗场景。

验证反馈与历史记录 验证结果（包括观察描述、是否应当重试、回退操作、替代操作）被记录在 `StepRecord.verification` 字段中。下一轮推理时，`build_som_user_prompt` 将最近步骤的验证结果格式化为 `[OK]`（验证通过）、`[NO EFFECT]`（操作无效）或 `[FAIL]`（执行失败）标记，连同失败原因一起展示在历史记录中。当连续出现 2 次以上失败标记时，`prompt` 中额外添加警告信息，引导 VLM 尝试完全不同的操作策略。

1.5 前端 Web 交互界面

前端基于 FastAPI WebSocket 与纯 HTML/CSS/JavaScript 构建，采用玻璃态深色主题。核心功能包括任务配置、Agent 步骤实时展示、浏览器画面实时流和键鼠交互反馈。

实时截图流（CDP Screencast） 前端通过独立 WebSocket 通道 `/ws/screencast` 接收浏览器实时画面流。后端利用 Playwright 的 Chrome DevTools Protocol（CDP）接口启动 `Page.startScreencast`，以 960×540 分辨率、约 15fps 的频率持续截取当前标签页画面，通过专用 WebSocket 推送至前端渲染到 Canvas 元素上。该通道与控制消息 WebSocket 完全分离，避免了高频帧数据阻塞控制消息的传输。

Screencast 模块在浏览器切换标签页时自动检测并重建 CDP 会话——当帧超时或会话断开时，通过 `env.get_cdp_session()` 重新获取新标签页的 CDP 连接并重启推流，确保实时画面始终跟随 Agent 当前操作的标签页。

键鼠交互反馈 前端 Canvas 元素接收用户的键盘和鼠标事件（`click`、`wheel`、`keydown` 等），将事件类型与坐标通过主 WebSocket `/ws/agent` 以 `browser_input` 消息发送至后端。后端通过 CDP 的 `Input.dispatchMouseEvent`、`Input.dispatchKeyEvent` 等接口将用户操作转发至浏览器，实现用户在 Web 界面上直接操控 Agent 浏览器窗口的能力。

2. HTML 文本输入模式

HTML 模式是为 Mind2Web 数据集评测而引入的纯文本输入变体。其设计目标是让同一个 Agent 框架能够在不依赖截图的情况下，仅通过 HTML 元素文本列表完成网页操作评估。由于 Mind2Web 数据集预先标注了每个步骤的正负候选元素及其 `backend_node_id`，HTML 模式不需要截图和 SoM 标注，改为直接从数据集读取候选列表并格式化为编号文本供 LLM 选择。

HTML 模式与 SoM 模式共享完全相同的 Agent 循环和 Executor，差异仅在于 Perceptor 和 Prompt。如图2所示：

- **感知层：**由 HTMLPerceptor 替代 SoMPerceptor，离线评估时读取 Mind2Web 候选数据，实时运行时可回退至 ElementExtractor 提取真实页面元素，同时提取 `body.innerText` 作为页面内容摘要供 LLM 理解当前页面状态。
- **推理层：**Reasoner 自动检测 `perception.screenshot` 是否为空，若为空则切换至 HTML 文本 Prompt，将元素列表、页面文本、历史记录和当前计划格式化为纯文本发送给 LLM。
- **验证层：**HTML 模式下无截图，验证功能被简化为 URL 变化检测和 DOM 错误弹窗扫描。

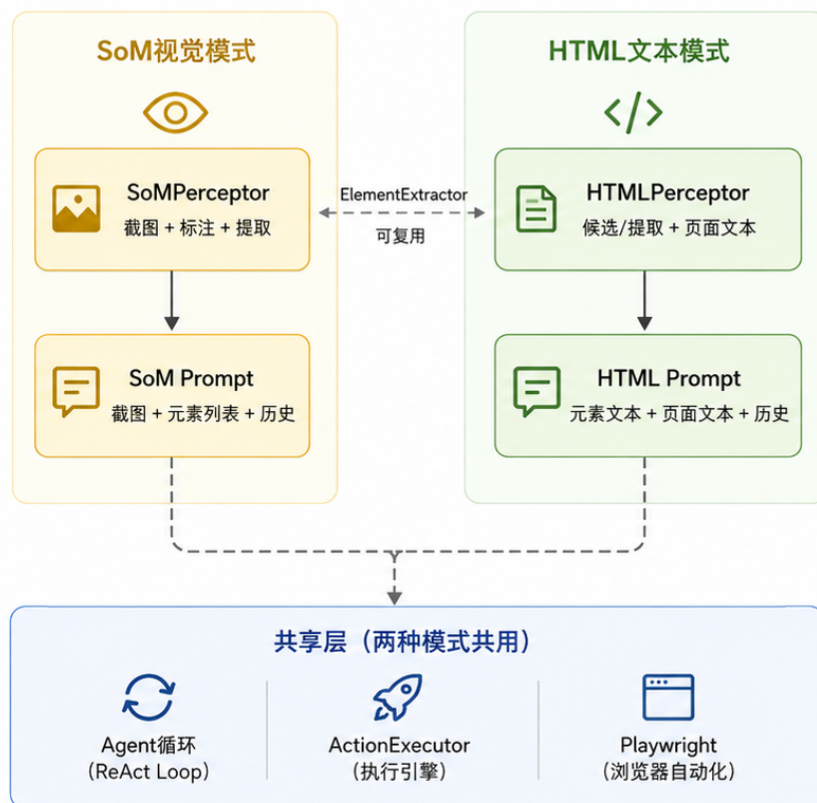


图 2: SoM 模式与 HTML 模式的架构对比

HTML 模式无需额外开发，仅需替换 Perceptor 和 Prompt 即可从视觉输入切换为文本输入，有效降低了为不同数据集定制评测流程的成本。

四 实验评测与分析

针对项目模型的 SoM 输入与 HTML 输入模式分别设计了评测实验。所有本地模型均在一台 RTX 3090 24GB ×1, 内存 60GB 的计算机上运行，并使用 Qwen 3VL-8B-Instruct 作为本地测试模型，使用 API 调用 Qwen3.7-max 测试。选取任务成功率 (SR)、元素准确率 (Ele. Acc)、任务单步操作的 F1 分数 (OP.F1)、任务单步成功率 (Step SR) 以及单步平均耗时作为评价 Web Agent 性能的标准。其中，Ele. Acc 表示各任务内选择正确网页元素的步数比例跨任务的均值，Op.F1 分数表示单步操作的 F1 值，Step SR 表示各任务内元素与操作均正确的步数比例跨任务的均值，SR 表示所有步骤均成功执行的任务数占数据集总任务数的比例。

$$\begin{aligned}
\text{Ele.Acc} &= \frac{1}{M} \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} \mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \\
\text{Op.F1}_{i,j} &= \mathbb{I}(\hat{op}_{i,j} \in S_{op}(op_{i,j})) \times \begin{cases} 1 & op_{i,j} = \text{CLICK} \\ \text{F1}(v_{i,j}, \hat{v}_{i,j}) & op_{i,j} \in \{\text{TYPE}, \text{SELECT}\} \end{cases} \\
\text{Step SR} &= \frac{1}{M} \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} [\mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \times \mathbb{I}(\text{Op.F1}_{i,j} = 1)] \\
\text{SR}_i &= \prod_{j=1}^{N_i} [\mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \times \mathbb{I}(\text{Op.F1}_{i,j} = 1)]
\end{aligned}$$

其中 M 为数据集内的总任务数, N_i 为第 i 个任务的步骤数, $e_{i,j}, \hat{e}_{i,j}$ 分别为第 i 个任务第 j 步中 Ground Truth 的等价网页元素集合和 Agent 预测的网页元素, $op_{i,j} \in \{\text{CLICK}, \text{TYPE}, \text{SELECT}\}$ 为 Ground Truth 操作类型, $\hat{op}_{i,j}$ 为预测操作类型, $v_{i,j}, \hat{v}_{i,j}$ 分别为 Ground Truth 和预测的操作值。 $S_{op}(\text{CLICK}) = \{\text{CLICK}, \text{HOVER}, \text{PRESS}\}$ 为 Mind2Web 中将 HOVER 和 PRESS_ENTER 映射至 CLICK 的等价操作集合; $\text{F1}(\cdot, \cdot)$ 为 F1 分数。

1. HTML 输入模式下的性能测评

选用 Mind2Web 数据集对模型进行评测。Mind2Web 数据集是一个大规模的 Web Agent 基准数据集, 将真实世界的静态网页作为数据, 具有 1,341 个不同任务和 46 步的最长任务步数, 在 Web Agent 领域被广泛用于测试模型的页面理解与执行能力。

我们对 Mind2Web 数据集提供的评测方法进行了修改。在 Mind2Web 数据集提供的 HTML 应用过滤规则只保留可交互元素后, 使用一个基于 Bert2 实现的预训练模型 all-MiniLM-L6-v2 对剩余候选元素进行排序并交给 VLM 进行选择, 并给出对应的操作类型和元素, 最后计算 Ele. Acc, OP.F1, Step SR 和 SR。

表 1: HTML 输入模式下模型在 Mind2Web 数据集上的性能

	Cross-Task				Cross-Website				Cross-Domain				Time(s)
	EA	OP	SS	SR	EA	OP	SS	SR	EA	OP	SS	SR	
Qwen3-VL-8B	41.9	84.0	33.9	1.5	32.0	78.0	26.4	1.0	36.0	83.0	28.0	3.0	1.3
Qwen3.7-max	64.6	80.5	52.0	8.7	57.9	81.3	42.4	6.2	55.6	81.9	43.8	7.3	14.3

由表 1 可以看出, Qwen3.7-max 在元素定位上显著优于 Qwen3-VL-8B, 元素准确率平均比 Qwen3-VL-8B 提高了 22.7%, 最终导致更高的单步成功率与任务成功率。但 Qwen3-VL-8B 作为小模型具有快速响应与低成本的优势, 在相对较好的网页操作性能和较快的响应时间中取得了平衡。Qwen3.7-max 的参数量过大, 虽然准确率更高但响应时间是 Qwen3-VL-8B 的 11 倍, 不具有实时性。

2. SoM 输入模式下的性能测评

为评估 SoM 视觉输入模式在真实网页场景下的表现, 自行设计了包含 30 个任务的测试集, 全部基于可公开访问的真实网站。测试任务覆盖多种难度与应用场景, 代表性任务包括: 搜索”中山大学广州校区南校园”并返回地址、打开哔哩哔哩首页点击第一个视频后返回作者头像、登录 QQ 邮箱并向指定地址发送端午祝福邮件等。完整任务列表见附录。所有测试均使用 Qwen 3VL-8B-Instruct 模型, 在有头浏览器模式下运行, 每个任务至少重复运行 10 次以减小随机性。

由于自建测试集缺少 Mind2Web 数据集集中的 Ground Truth 标注(如真实操作序列和正负候选元素), 无法直接计算 Ele. Acc 和 Op.F1 等微观指标。因此针对真实网页场景, 定义以下两项宏观指标评价 Agent 的任务完成能力:

$$SR = \frac{S}{N}$$

$$AvgSteps = \frac{1}{S} \sum_{i \in S} steps_i$$

其中 $N = 30$ 为测试任务总数, S 为成功完成 (Agent 返回正确答案且人工验证通过) 的任务数, S 为成功任务集合, $steps_i$ 为第 i 个任务成功完成时使用的步数。

表 2: SoM 输入模式与 HTML 输入模式在自建测试集上的对比

模型	SoM 输入		HTML 输入	
	SR	AvgSteps	SR	AvgSteps
Qwen 3VL-8B	73.3	10.5	40.0	6.8

由表 2 可以看出, SoM 视觉输入模式相比 HTML 文本输入模式在真实网页任务上具有显著优势。任务成功率从 40.0% 提升至 73.3%, 提升幅度达 83.3%。分析任务完成

情况发现，HTML 输入模式能够成功完成的任务以简单的搜索引擎查询为主（如事实类搜索、天气查询等），一旦任务涉及表单填写、页面跳转、多步交互等复杂操作，成功率显著下降。SoM 模式由于能够直观地识别页面布局与元素空间位置，在需要点击特定按钮、填写表单、切换标签页等复杂操作时表现更好。但 SoM 模式的完成任务平均步数 10.5 步明显高于 HTML 模式的 6.8 步，这一方面是因为 SoM 模式成功完成的任务本身更复杂，自然需要更多操作步数；另一方面也与 SoM 模式下模型倾向于执行更多探索性操作（如下拉滚动、尝试不同元素）有关。

总体而言，SoM 输入模式以更多的操作步数为代价，换取了更高的任务完成率与更广泛的任务适用性，在真实网页场景下优于 HTML 文本输入模式。

五 参考文献

参考文献

- [1] He, Y., et al. “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models.” 10.48550/arXiv.2401.13919
- [2] Microsoft Research. “Fara-7B: A Foundation Model for GUI Automation.” 2025.
- [3] Xinyi Li, et al. “A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges” Computer Science.10.1007/s44336-024-00009-2
- [4] Yang, J., et al. “Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V.” arXiv preprint arXiv:2310.11441, 2023.
- [5] Cheng, K., et al. “R-VLM: Region-Aware Vision Language Model for Precise GUI Grounding.” arXiv preprint 10.48550/arXiv.2507.05673.
- [6] Furuta, H., et al. “WebGUM: Multimodal Web Navigation with Instruction-Finetuned Foundation Models.” arXiv.2305.11854v4
- [7] Microsoft. “Playwright: Fast and Reliable Cross-Browser Automation.” <https://playwright.dev/>, 2025.
- [8] OpenClaw Contributors. “OpenClaw: Open-Source Multimodal Web Agent.” GitHub, 2025.

- [9] SMART-Lab Purdue. “SMART-LLM: Multi-Agent Task Planning.” GitHub, 2024.